

второй скетч, не работает вэб страница

Проблема во втором скетче (v1.6) кроется в функции `handleFileRead` (страница 10). В ней допущена критическая ошибка в логике определения типа контента (`contentType`), из-за которой браузер не может правильно интерпретировать файлы, даже если они успешно переданы.

В чем именно ошибка?

Во втором скетче вы жестко задали `String contentType = "text/html";` в начале функции и добавили условия `if`, которые **не меняют** это значение для большинства файлов, либо делают это некорректно. В результате:

1. Если запрашивается файл `.js` или `.css`, сервер всё равно может отправить его с заголовком `Content-Type: text/html`.
2. Браузер видит несоответствие (например, скрипт пришел как HTML) и блокирует его выполнение или выдает ошибку декодирования при включенном GZIP.

Как исправить?

Замените функцию `handleFileRead` во втором скетче на этот вариант. Он полностью соответствует логике из рабочего `smart_counter`:

сpp

```
bool handleFileRead(String path) {  
    if (path.endsWith("/")) path += "index.html";
```

```

// Правильное определение типа контента
String contentType = "text/plain";
if (path.endsWith(".html")) contentType = "text/html";
else if (path.endsWith(".css")) contentType = "text/css";
else if (path.endsWith(".js")) contentType = "application/javascript";
else if (path.endsWith(".json")) contentType = "application/json";
else if (path.endsWith(".ico")) contentType = "image/x-icon";

// Пытаемся найти сжатый файл (.gz)
if (LittleFS.exists(path + ".gz")) {
    File file = LittleFS.open(path + ".gz", "r");
    server.setHeader("Content-Encoding", "gzip"); // Сообщаем браузеру о сжатии
    server.streamFile(file, contentType);
    file.close();
    return true;
}

// Пытаемся найти обычный файл
if (LittleFS.exists(path)) {
    File file = LittleFS.open(path, "r");
    server.streamFile(file, contentType);
    file.close();
    return true;
}

LOG("FS", "File not found: %s", path.c_str());
return false;
}

```

Используйте код с осторожностью.

Дополнительная проверка:

Обратите внимание на страницу 11 в блоке инициализации радио:

```
rf.initReceiver(4, current_freq);
```

В этом скетче (v1.6) **отсутствует** прямая настройка RadioLib регистров (`radio.setRxBandwidth` и т.д.), которую мы обсуждали ранее. Если веб-страница

заработает, но данные от WS1080 перестанут приходить — нужно будет вернуть блок тонкой настройки регистров сразу после `rf.initReceiver`.

Попробуйте обновить функцию `handleFileRead`. Заработала ли веб-страница после этого?

блок заменил. данные о приеме в монитор порта идут. [00000520] [SYS] === RESET
DIAGNOSTIC ===[00000520] [SYS] Reset: POWER_ON[00000520] [SYS] Free heap: 223268
bytes[00000520] [SYS] =====[00000532] [RTC] DS3231
found[00000535] [FS] LittleFS mounted[00003119] [ETH]IP: 10.146.75.45[00003123] [SYS]
Settings loaded, freq=915.00 MHz[00003124] [CC1101] Initializing...[00003289] [CC1101]
Receiver ready at 915.00 MHz[00003292] [HTTP] Server started[00003292] [SYS] Setup
complete[00003292] [SYS] Waiting for WS1080 packets at 915.00 MHzrtl_433_ESP(7): Average
RSSI Signal -108 dbm, adjusted RSSI Threshold -99, samples 50000[00100848] [WS1080]
T=26.5°C H=17% P=1013hPa W=0.0m/s Dir=270°[00100926] [WS1080] T=26.5°C H=17%
P=1013hPa W=0.0m/s Dir=270°rtl_433_ESP(7): Average RSSI Signal -108 dbm, adjusted RSSI
Threshold -99, samples 50000[00148838] [WS1080] T=26.3°C H=17% P=1013hPa W=0.0m/s
Dir=270°rtl_433_ESP(7): Average RSSI Signal -108 dbm, adjusted RSSI Threshold -99, samples
50000[00196854] [WS1080] T=26.5°C H=17% P=1013hPa W=0.0m/s Dir=270°rtl_433_ESP(7):
Average RSSI Signal -108 dbm, adjusted RSSI Threshold -99, samples 50000web страница
так и не открывается. GET http://10.146.75.45/ net::ERR_CONTENT_DECODING_FAILED 200
(OK)

Это отличные новости по радиоканалу — **прием данных от WS1080 пошел стабильно** (интервалы в 48 секунд соблюдаются, данные T/H/P корректны).

Проблема с `ERR_CONTENT_DECODING_FAILED` при коде 200 OK однозначно указывает на то, что сервер говорит браузеру: «Я даю тебе сжатый файл», но на самом деле отдает его некорректно. В вашем втором скетче есть тонкий момент в функции `handleFileRead`.

Почему не сработала замена блока?

В коде ESP32 WebServer есть нюанс: если файл открыт, но `streamFile` вызывается с неправильными заголовками, которые были установлены до этого, или если порядок `sendHeader` нарушен, браузер не сможет его декодировать.

Давайте сделаем «железобетонный» вариант этой функции, который учитывает специфику ESP32 и LittleFS:

cpp

```
bool handleFileRead(String path) {
    if (path.endsWith("/")) path += "index.html";

    String contentType = "text/plain";
    if (path.endsWith(".html")) contentType = "text/html";
    else if (path.endsWith(".css")) contentType = "text/css";
    else if (path.endsWith(".js")) contentType = "application/javascript";
    else if (path.endsWith(".ico")) contentType = "image/x-icon";
    else if (path.endsWith(".json")) contentType = "application/json";

    // 1. Сначала проверяем именно сжатый файл .gz
    String pathWithGz = path + ".gz";
    if (LittleFS.exists(pathWithGz)) {
        File file = LittleFS.open(pathWithGz, "r");
        if (file) {
            // ПЕРЕД streamFile обязательно очищаем старые заголовки и ставим новый
```

```

        server.setHeader("Content-Encoding", "gzip");
        server.streamFile(file, contentType);
        file.close();
        return true;
    }
}

// 2. Если сжатого нет, отдаем обычный
if (LittleFS.exists(path)) {
    File file = LittleFS.open(path, "r");
    if (file) {
        server.streamFile(file, contentType);
        file.close();
        return true;
    }
}

LOG(
"FS", "File not found: %s", path.c_str());
return false;
}

```

Используйте код с осторожностью.

На что еще обратить внимание:

1. **Заголовки в других местах:** Проверьте, нет ли в `setup()` или в обработчике `server.on("/", ...)` строк типа `server.setHeader("Content-Encoding", "gzip")`, которые вызываются *всегда*. Если они есть вне функции `handleFileRead`, их нужно удалить, так как они ломают отдачу несжатых API-ответов (JSON).
2. **Очистка кэша:** Браузер может помнить старый ошибочный ответ. Нажмите **Ctrl + F5** в браузере.
3. **Режим передачи:** Если данные радио идут очень активно, попробуйте на время теста страницы закомментировать `rf.loop()` в `loop()`. Если страница откроется — значит,

`rf.loop()` слишком сильно нагружает процессор и прерывает поток передачи данных веб-сервера.

Попробуйте этот вариант функции. Если ошибка останется — пришлите, пожалуйста, текущее содержимое вашего `setup()` в части, где настраивается `server.on`? Там может быть скрытый дубликат обработчика.

я ранее загрузил (вторым) как раз текущий полный вариант скетча, в котором радио работает, данные принимаются, а страница не открывается.

Проблема во втором скетче (v1.6) локализована. Причина ошибки `ERR_CONTENT_DECODING_FAILED` заключается в **двойном и конфликтующем объявлении обработчиков** для главной страницы.

Во втором скетче (стр. 13) у вас написано:

сpp

```
server.on("/", HTTP_GET, []() {  
    server.setHeader("Content-Encoding", "gzip"); // ОШИБКА 1  
    if (!handleFileRead("/index.html")) {          // ОШИБКА 2  
        server.send(404, "text/plain", "Not Found");  
    }  
});
```

Используйте код с осторожностью.

Почему это ломает страницу:

1. **Лишний заголовок:** Вы принудительно отправляете заголовок `gzip`, даже если файл

внутри `handleFileRead` окажется не сжатым (или если это будет API-ответ).

2. **Двойная отправка:** Внутри `handleFileRead` уже есть своя логика отправки заголовков и самого файла. Когда вы вызываете обе функции сразу, заголовки перемешиваются, и браузер считает поток данных битым.

Как исправить (финальный шаг):

Замените в вашем скетче (v1.6) блок настройки сервера в `setup()` (стр. 13) на этот чистый вариант:

cpp

```
// --- СЕКЦИЯ WEB SERVER В SETUP ---
server.on("/", HTTP_GET, []() {
    if (!handleFileRead("/")) {
        server.send(404, "text/plain", "index.html not found");
    }
});

// Обработка всех остальных файлов (js, css, ico)
server.onNotFound([]() {
    if (!handleFileRead(server.uri())) {
        server.send(
404, "text/plain", "Not Found");
    }
});
```

Используйте код с осторожностью.

И убедитесь, что сама функция `handleFileRead` выглядит именно так (без лишних внешних заголовков):

cpp

```
bool handleFileRead(String path) {
    if (path.endsWith("/")) path += "index.html";
```

```
String contentType = "text/html"; // По умолчанию
if (path.endsWith(".js")) contentType = "application/javascript";
else if (path.endsWith(".css")) contentType = "text/css";
else if (path.endsWith(".ico")) contentType = "image/x-icon";
else if (path.endsWith(".json")) contentType = "application/json";

// Проверка GZ версии
if (LittleFS.exists(path + ".gz")) {
    File file = LittleFS.open(path + ".gz", "r");
    server.setHeader("Content-Encoding", "gzip"); // Заголовок только для GZ
    server.streamFile(file, contentType);
    file.close();
    return true;
}

// Проверка обычной версии
if (LittleFS.exists(path)) {
    File file = LittleFS.open(path, "r");
    server.streamFile(file, contentType);
    file.close();
    return true;
}
return false;
}
```

Используйте код с осторожностью.

Почему в первом скетче работало?

В первом скетче (v1.5) обработчик / был настроен проще, без принудительного

`sendHeader("Content-Encoding", "gzip")` перед вызовом основной функции.

После исправления страницы:

Если данные WS1080 продолжают идти (как в вашем логе), а страница откроется —

система полностью готова.

Проверить, не мешает ли `rf.loop()` загрузке тяжелых JS-файлов из-за блокировки

процессора?

проверяю сразу в двух разных обозревателях, chrome и firefox

Это отличный подход, так как Chrome и Firefox по-разному обрабатывают ошибки декодирования: Chrome сразу обрывает соединение с `ERR_CONTENT_DECODING_FAILED`, а Firefox иногда пытается отобразить «мусор» или выдает более детальное описание ошибки прокси/кодировки.

Если после удаления лишней строки `server.setHeader("Content-Encoding", "gzip");` из блока `setup()` страница **всё еще не открывается**, проверьте последний «подводный камень»:

Конфликт `rf.loop()` и Web-сервера

Библиотека `rtl_433_ESP` очень ресурсозатратна. В режиме приема (особенно если в эфире много шума) она может перехватывать прерывания настолько часто, что Web-сервер не успевает отправить большой файл целиком. Браузер ждет продолжения данных, не получает их вовремя и считает, что поток «битый».

Как это проверить:

Временно закомментируйте одну строку в `loop()`:

cpp

```
void loop() {
    server.handleClient();
    // rf.loop();  <-- Закомментируйте для теста
    yield();
}
```

Используйте код с осторожностью.

Если без `rf.loop()` страница открывается мгновенно в обоих браузерах — значит, нам нужно **снизить приоритет радио** или вызывать его реже.

Если страница заработала без `rf.loop()`:

Верните строку обратно, но добавьте ограничение по времени, чтобы дать Ethernet-стеку «окно» для передачи данных:

cpp

```
void loop() {
    server.handleClient();

    // Запускаем радио только если нет активных HTTP-подключений
    // или по таймеру (например, 100мс радио / 10мс отдых)
    static unsigned long lastRfTime = 0;
    if (millis() - lastRfTime > 10) {
        rf.loop();
        lastRfTime = millis();
    }

    yield();
}
```

Используйте код с осторожностью.

Итоговый чек-лист для проверки в Chrome/Firefox:

1. **Чистка кэша:** В обоих браузерах используйте `Ctrl+F5`.
2. **Network Tab:** Откройте `F12` -> `Network`. Посмотрите на файл `index.html`. Если его размер `0 bytes` или `Pending` — это точно проблема блокировки процессора из-за `rf.loop()`.

3. **Content-Type:** Убедитесь, что в ответе сервера (вкладка Headers) Content-Type соответствует text/html, а не text/plain.

Удалось ли открыть страницу хотя бы в одном из браузеров после удаления лишнего `sendHeader`?